

T05–T06 — Sintassi Java, Strutture di Controllo e Classi Base

Costrutti Condizionali e Iterativi, Definizione di Classe e Nascita del Progetto SimpleDate

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo in dettaglio la sintassi fondamentale di Java ([T05 - Java Basic Syntax.pdf](#)), la struttura delle classi e degli oggetti ([T06 - Java Classes and Objects.pdf](#)) e il codice sorgente di debutto del corso: il progetto [SimpleDate](#) e il file di laboratorio [StringPlayground.java](#).

0.0.1 1. Teoria: Concetti Teorici dalle Slide

A. Sintassi di Base e Costrutti Semplici (Slide T05, 1–19)

- **Variabili e Inizializzazione Automatica:**
 - Una variabile associa un nome e un tipo a una locazione di memoria.
 - **Regola cruciale:** Solo i campi di classe/istanza (*fields*) vengono inizializzati automaticamente al default (0, 0.0, false, '\u0000', null). Le variabili locali nello Stack richiedono l'assegnamento esplicito prima dell'uso.
- **Costanti (static final):**
 - `final static int I = 5;` la variabile è a sola lettura; deve essere inizializzata contestualmente o nel costruttore e non può più essere riassegnata.
- **Identificatori e Convenzioni di Naming:**
 - Regola sintattica: sequenza arbitraria di caratteri [a-z], [A-Z], [0-9], _ e \$, che non inizi con una cifra e non coincida con una delle **50 parole chiave riservate** (class, static, new, this, super, switch, goto, const, ecc.).
 - Convenzioni ufficiali Java:
 - * **Classi:** UpperCamelCase (es. SimpleDate, MainDate).
 - * **Metodi, variabili locali, campi e parametri:** lowerCamelCase (es. daysPerMonth, myCar).
 - * **Costanti:** UPPER_SNAKE_CASE (es. FORMAT, PI_CONST).
- **Caratteri di Escape Speciali:**
 - \n (LF linefeed, \u000a), \r (CR carriage return, \u000d), \t (tab orizzontale, \u0009), \b (backspace, \u0008), \f (form feed, \u000c), \" (doppio apice), \' (singolo apice), \\ (backslash).
- **Commenti e Documentazione:**
 - // ... (singola riga).

- `/* ... */` (multi-riga).
- `/** ... */` (**Javadoc**): elaborato dal tool `javadoc` per generare automaticamente la documentazione HTML delle API.

- **Espressioni, Valutazione Sinistra-Destra e Side Effects:**

- In Java le sotto-espressioni sono valutate **rigorosamente da sinistra a destra**:
 - * `int i = 1; int j = (i = 2) * i;` $\Rightarrow$ `i` diventa 2, poi moltiplicato per `i` (2) $\Rightarrow$ `j` = 4.
 - * `int t = 3; eval(t + (t = 2))` $\Rightarrow$ prima si valuta `t` (3), poi `(t = 2)` (2) $\Rightarrow$ `3 + 2` = 5.

- **Operatori Aritmetici Unari: Pre-incremento vs Post-incremento:**

- `x++` (post-incremento): restituisce il valore attuale di `x` e *successivamente* incrementa `x` di 1.
- `++x` (pre-incremento): incrementa *subito* `x` di 1 e restituisce il nuovo valore.

- **L'esercizio tranello delle slide:**

```
1 int t = 3;
2 t = -t++ - ++t; // Quanto vale t?
```

- * Valutazione da sinistra a destra:

1. `-t++`: valuta `-3`, e incrementa `t` a 4.
2. `++t`: incrementa subito `t` da 4 a 5, e valuta 5.
3. Operazione: `(-3) - 5` = `-8`. Assegnato a `t`, il valore finale è **-8**.

- **Operatore Ternario:** condizione ? `expr1` : `expr2`.

- **Operatori Logici e Corto-Circuito:**

- `&`, `|`: valutano sempre entrambi i rami.
- `&&`, `||`: valutazione a corto circuito (*short-circuit*). Se il primo operando determina già il risultato, il secondo non viene nemmeno eseguito (essenziale per prevenire crash su puntatori nulli).

- **Assegnamenti Composti:** `x += expr` equivale a `x = x + (expr)`. Se `x = 4`, l'espressione `x+ = x - 7` assegna `x = 4 + (4 - 7) = 1`.

B. Costrutti di Controllo di Flusso (Slide T05, 20–29)

- **Blocchi** `{ ... }`: delimitano lo scope di vita delle variabili locali.

- **Condizionali:** `if (Bexpr) ... else ...`.

- **Costrutto switch-case e il Fall-Through:**

- Seleziona il ramo da eseguire in base al valore di un'espressione (`byte`, `short`, `char`, `int`, `String`, `enum`).
- Se non si inserisce l'istruzione `break`, l'esecuzione **continua nei rami successivi** (*fall-through*). Questo meccanismo, se usato volontariamente, permette di raggruppare casi che condividono la stessa logica (come vedremo in `daysPerMonth`).

- **Cicli:**

- `while (cond){ ... }`: test preliminare (può eseguire 0 volte).
- `do { ... } while (cond);`: test posticipato (esegue almeno 1 volta).
- `for (init; cond; step){ ... }`: ciclo controllato. Supporta inizializzazioni e passi multipli separati da virgola:

```
1 for (int i = 0, j = 9; i <= 9 && j >= 0; i++, j--) { ... }
```

- **Salto di Controllo:**

- `break`: interrompe il ciclo o lo switch ed esce dal blocco.
 - `continue`: salta immediatamente alla successiva iterazione del ciclo.
 - `return`: esce dal metodo corrente restituendo eventualmente un valore.
-

C. Classi, Oggetti, `this` e `toString` (Slide T06, 1–28)

- **Perché modellare una data? (La vignetta a lezione, Slide 7):**

- Lo slide show mostra il fumetto geek: «*Imagine an object much like a thing in the real world, like... A DATE?*». È da qui che nasce la classe `Date` come filo conduttore del corso.

- **Anatomia di una Classe:**

- **Campi d'istanza (*Fields*):** variabili che memorizzano lo stato di ciascun oggetto.
- **Metodi d'istanza (*Methods*):** funzioni che definiscono il comportamento e i messaggi accettati dall'oggetto.
- **Firma del metodo:** `tipoRitorno nomeMetodo(tipoParametro nomeParametro, ...)`. Se non restituisce nulla si usa `void`.
- **Overloading:** possibilità di definire più metodi con lo stesso nome purché differiscano per numero, tipo o ordine dei parametri.

- **Entrypoint `main` e Metodi Statici (`static`):**

- `public static void main(String[] args)`: il launcher della JVM non passa il nome del programma in `args[0]` (a differenza del C). `args[0]` è già il primo parametro utente.
- **Metodi Statici:** appartengono alla classe e non alle singole istanze. Si invocano con `NomeClasse.metodo()`. Non possono accedere allo stato d'istanza né usare `this`.

- **Creazione dell'Oggetto (`new`) e Costruttore:**

- L'istruzione `new Date(...)`:
 1. Alloca memoria nello Heap per l'oggetto.
 2. Inizializza i campi ai valori di default.
 3. Invoca il **costruttore** (metodo speciale con lo stesso nome della classe e senza tipo di ritorno) per valorizzare lo stato.
 4. Restituisce il riferimento all'oggetto appena creato.

- **Aliasing e Garbage Collection:**

- Se scriviamo `Date d2 = d1;`, le due variabili puntano allo stesso identico oggetto nello Heap (*aliasing*).
- Quando una reference viene azzerata (`d1 = null; d2 = null;`), l'oggetto diventa orfano e il Garbage Collector può bonificarlo.

- **La Parola Chiave `this`:**

- È il riferimento implicito all'oggetto corrente su cui è stato invocato il metodo.
- È **obbligatoria** quando i parametri del costruttore/metodo hanno lo stesso identico nome dei campi d'istanza (*shadowing dei parametri*):

```
1 public Date(int day, int month, int year) {
2     this.day = day;      // this.day è il campo dell'oggetto nello Heap
3     this.month = month;  // day è il parametro locale nello Stack
4     this.year = year;
5 }
```

- **Rappresentazione a Stringa e toString():**

- Tutte le classi Java ereditano dalla superclasse universale `java.lang.Object` il metodo `public String toString()`.
- Il default di `Object.toString()` stampa `NomeClasse@IndirizzoHashCode` (es. `Date@5acf9800`).
- La JVM invoca **implicitamente** `toString()` ogni volta che un oggetto viene concatenato a una stringa ("Oggi: " + today).
- Sovrascrivendo (*overriding*) `public String toString()` nella classe, possiamo personalizzare la stampa dell'oggetto.

0.0.2 2. Codice: Analisi del Codice Ufficiale del Docente

1. SimpleDate/src/Date.java

```
1 public class Date {
2     // 1. Stato dell'oggetto: campi di istanza (visibilità default di package per ora)
3     int day;
4     int month;
5     int year;
6
7     // 2. Costante di classe condivisa da tutte le istanze
8     static final String FORMAT = "dd/mm/yyyy";
9
10    // 3. Costruttore: usa 'this' per distinguere campi da parametri
11    public Date(int day, int month, int year) {
12        this.day = day;
13        this.month = month;
14        this.year = year;
15        verify(); // Chiamata al metodo di validazione interna
16    }
17
18    // 4. Metodo di validazione dello stato
19    void verify() {
20        if (year < 0 || month < 1 || month > 12)
21            System.out.println("Illegal date!");
22        else
23            if (day < 1 || day > daysPerMonth(month))
24                System.out.println("Illegal date!");
25    }
26
27    String print() {
28        return day + "/" + month + "/" + year + " [" + FORMAT + "];"
29    }
30
31    // 5. Override di toString(): delega al metodo print()
32    public String toString() {
33        return print();
34    }
35
36    // 6. Metodo statico: calcolo puro senza bisogno di istanziare Date
37    static int daysPerMonth(int month) {
38        int days;
39        switch(month) {
40            case 4:
41            case 6:
42            case 9:
43            case 11:
44                days = 30; // Fall-through controllato per i mesi da 30 giorni
45                break;
46            case 2:
47                days = 28; // Febbraio fisso a 28 (il bisestile arriverà a Lezione 07!)
48                break;
```

```
49         default:
50             days = 31;
51             break;
52     }
53     return days;
54 }
55 }
```

Analisi: Dettagli Tecnici e Scelte del Docente:

1. **Perché daysPerMonth(month) è static?** Non ha bisogno di leggere this.day o this.year: riceve un mese intero e restituisce i giorni. Essendo una funzione pura di utilità, appartiene alla classe e può essere chiamata anche prima di creare oggetti (come fa il main).
2. **Lo switch con fall-through:** I case 4: case 6: case 9: case 11: sono privi di istruzione break intermedia; l'esecuzione scorre intenzionalmente fino a days = 30; break;.
3. **Validazione con verify():** In questa prima versione didattica, se la data è errata viene stampato a video "Illegal date!". Notare che l'oggetto non valido viene comunque creato in memoria! Solo più avanti (in Lezione 17 con le eccezioni) il docente mostrerà come impedire l'istanziamento lanciando un'eccezione.

2. SimpleDate/src/MainDate.java

```
1 import java.util.Scanner;
2
3 public class MainDate {
4     public static void main(String[] args) {
5         // Chiamata a metodo statico direttamente da classe
6         System.out.println(Date.daysPerMonth(4));
7
8         // Allocazione istanze nello Heap
9         Date today = new Date(14, 10, 2025);
10        Date tomorrow = new Date(15, 10, 2025);
11
12        System.out.println(today.print());
13        System.out.println(tomorrow.print());
14        System.out.println(today.toString());
15
16        String str = new String("ciao");
17        System.out.println(str.toString());
18
19        // Acquisizione input utente con java.util.Scanner
20        Scanner sc = new Scanner(System.in);
21        System.out.println("Enter the day: ");
22        int day = sc.nextInt();
23        System.out.println("Enter the month: ");
24        int month = sc.nextInt();
25        System.out.println("Enter the year: ");
26        int year = sc.nextInt();
27        sc.close(); // Chiusura dello stream di input
28
29        Date date = new Date(day, month, year);
30        System.out.println(date.toString());
31    }
32 }
```

3. StringPlayground.java Questo file illustra 3 concetti cardine sulle stringhe e l'ottimizzazione della memoria:

```
1 public class StringPlayground {
2     public static void main(String[] args) {
3         String empty = new String();
4         String str = "str";
5         System.out.println("len(empty): " + empty.length());
6         System.out.println("len(\"str\"): " + str.length());
7         System.out.println(str.charAt(1)); // Restituisce 't'
8
9         // str.charAt(1) = 'c'; // COMPILE-TIME ERROR: Le String in Java sono IMMUTABILI!
10
11        String same = new String("str");
12        System.out.println("str: " + str + " same: " + same);
13
14        // Confronto di puntatori (identità di memoria) vs contenuto semantico
15        System.out.println("str == same: " + (str == same));           // false! Oggetti diversi
16                                   nello Heap
17        String verySame = str;
18        System.out.println("str == verySame: " + (str == verySame)); // true! Puntano alla stessa
19                                   cella
20        System.out.println("str == same: " + str.equals(same));       // true! Stessi caratteri
21
22        // StringBuilder e Fluent Interface (Chaining)
23        int n = 5;
24        int sum = n * (n + 1) / 2;
25        StringBuilder sb = new StringBuilder();
26        sb.append("sum(").append(sum).append(")"); // Notazione a catena (fluent)
27        for (int i = 0; i < n; i++)
28            sb.append(" ").append(i);
29        sb.append(" ]");
30        System.out.println(sb.toString()); // sum(15)[ 0 1 2 3 4 ]
31    }
32 }
```

Analisi: Le Finezze di StringPlayground:

1. **Immutabilità:** in Java l'oggetto String non può essere modificato dopo la creazione. Non esiste un metodo `setCharAt()`.

2. **== vs .equals():**

- `str == same` confronta se le due variabili contengono lo **stesso indirizzo di memoria**. Restituisce false perché `same` è stato creato con `new String(...)` forzando una nuova allocazione nello Heap.
- `str.equals(same)` confronta i caratteri uno a uno e restituisce true.
- **Regola d'esame assoluta:** per confrontare gli oggetti e le stringhe si usa **sempre** `.equals()`, mai `==`!

3. **StringBuilder e Fluent Notation:**

- Ogni concatenazione con `+` su stringhe ordinarie crea un nuovo oggetto String temporaneo. Nei cicli questo genera un enorme sovraccarico di memoria per il Garbage Collector.
- `StringBuilder` alloca un buffer mutabile ridimensionabile internamente.
- Il metodo `.append()` restituisce un riferimento a `this` (l'istanza corrente del builder stesso), permettendo di concatenare le chiamate in cascata: `sb.append(...).append(...)` (**Fluent API / Method Chaining**).

0.0.3 3. Ciclo: Confronto con il tuo modulo es01

Nel tuo file `es01/src/main/java/es01/Date.java`: 1. **Switch Moderno vs Tradizionale**: Avevi già applicato la sintassi Java 14+ con la *Switch Expression*: `java static int daysPerMonth(int month){ return switch (month){ case 4, 6, 9, 11 -> 30; case 2 -> 28; default -> 31; }; }` Questa forma è sintatticamente pulitissima e idiomatica in Java moderno. 2. **Allineamento con la Lezione 05-06**: Per essere al 100% fedeli alla versione del docente: - Aggiungere la chiamata a `verify()` nel costruttore per il controllo di validità. - Correggere il refuso nella costante di formato: `"dd/mm/yyyy"` invece di `"dd/hh/yyyy"`.

Nota di Studio

Con le **Lezioni 05-06** abbiamo stabilito l'architettura iniziale di `Date`, lo scope, i metodi statici, le stringhe e `StringBuilder`.

Il prossimo blocco è la **Lezione 07 / Slide T07: Libraries and String Class**: - Evoluzione di `SimpleDate`: introduzione dell'algoritmo completo del **calendario Gregoriano per gli anni bisestili** (*leap year*). - Analisi della classe statica ausiliaria di calcolo matematico `java.lang.Math`. - Metodi avanzati della classe `String` (`substring`, `indexOf`, `split`, `trim`, `replace`). - Wrapper classes per i primitivi (`Integer`, `Double`, `Character`), *Autoboxing* e *Unboxing*.

Dammi conferma per procedere alla Lezione 07 / T07!